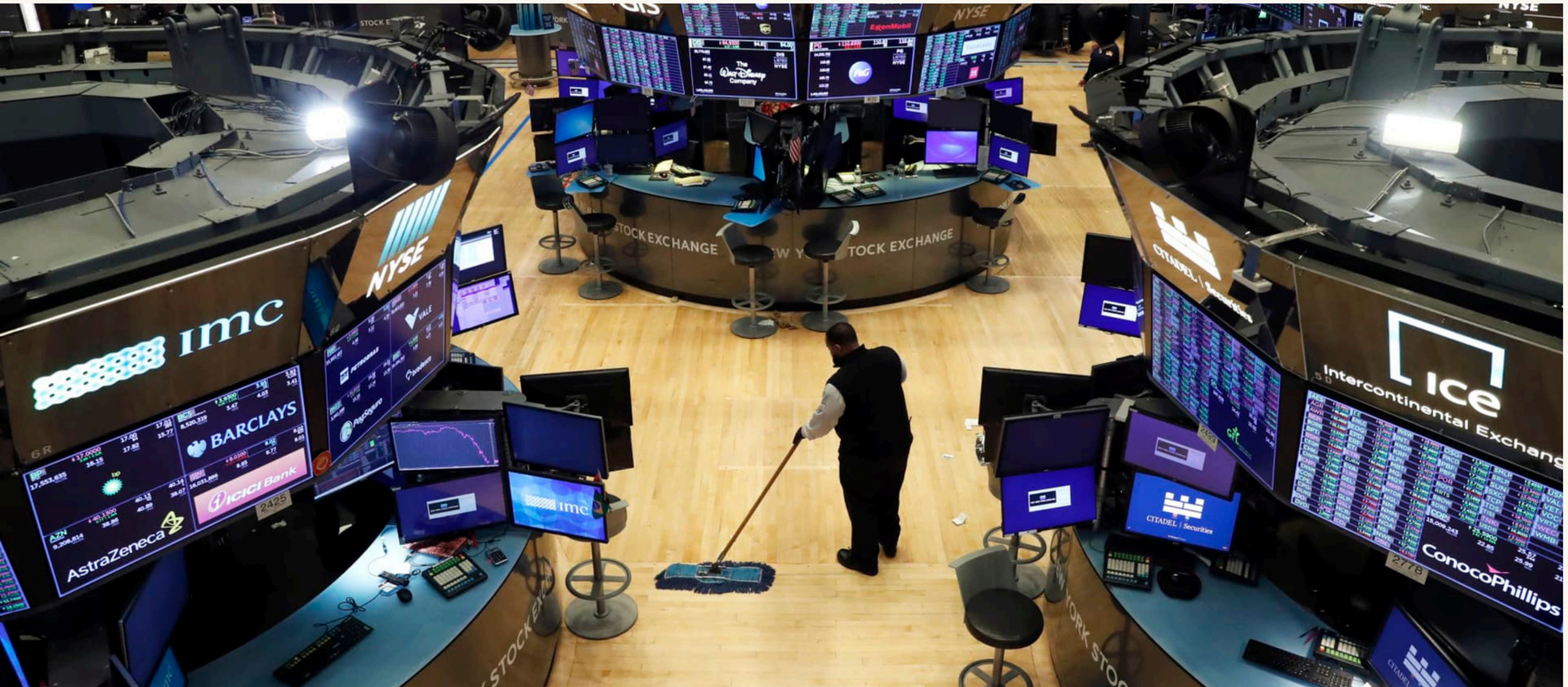
MAP: A Deterministic C++ Trading Engine



TEAM

Mingxuan Wang
Avi Maslow
Paul Kearns

Project Overview

Why Even Build an HFT Engine?

- Markets move too fast for humans to see what's actually happening.
- Because the market's behavior changes at microsecond scale.
- experiment — change latency, change strategies, stress-test the pipeline — and watch how the market react

Solution

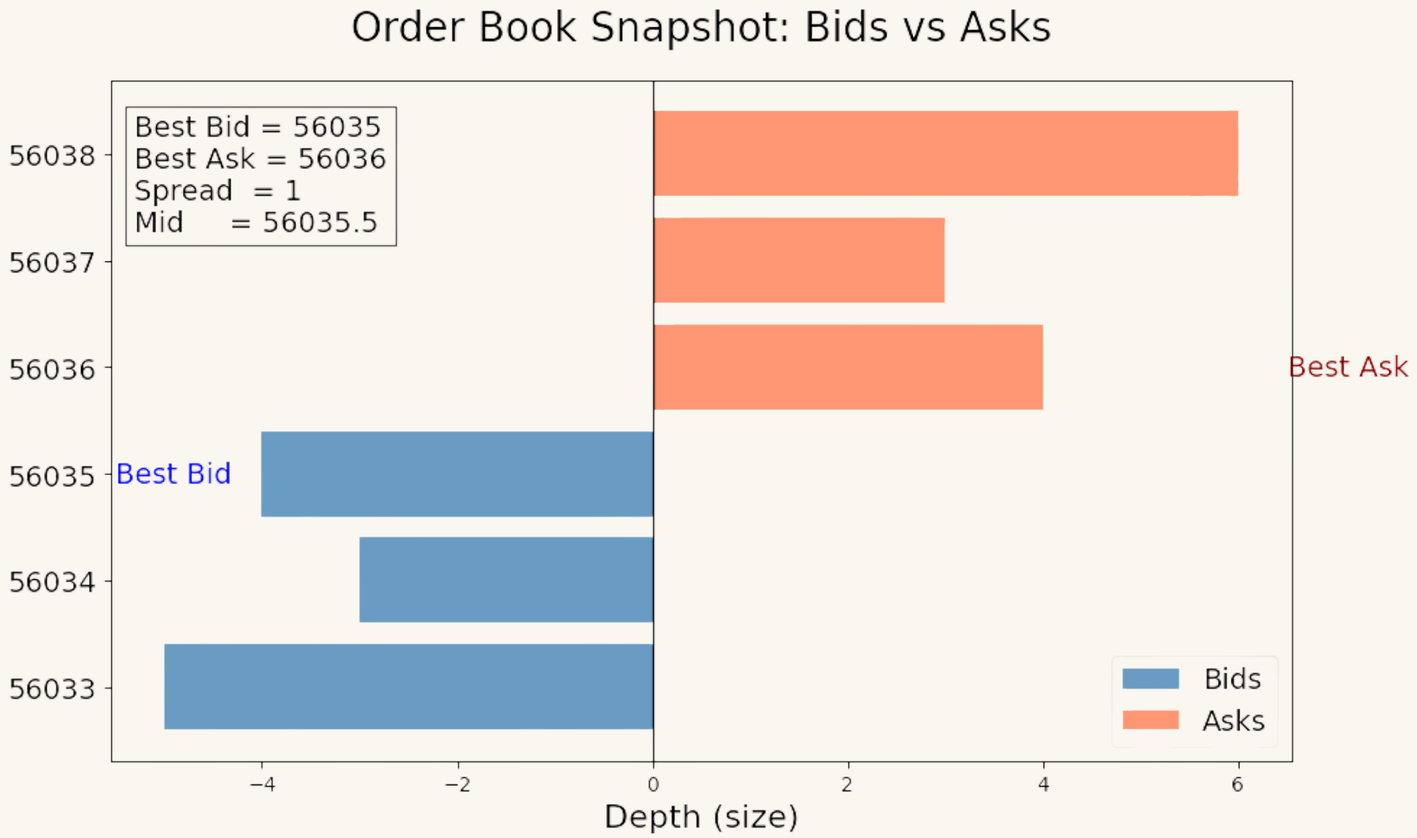
- Replay real BTC/ETH/ADA limit-order-book streams through a deterministic simulation engine.
- Event-driven pipeline combining Strategy, Risk, and OrderBook components, connected by producer→consumer queues.
- Deterministic logging (without it every test behaves differently)

Why It Matters

- Safe experimentation without money
- Controlled, reproducible strategy evaluation
- Microsecond-level insights unavailable from APIs
- Research framework similar to real HFT shops

Market Microstructure 101

- Best Bid – highest buy offer
- Best Ask – lowest sell offer
- Spread – ask − bid
- Mid-price – average of bid and ask
- Depth – quantity available at each price level
- Order Matching – how buys and sells get filled



What MAP Actually Does

- 1. Uses Real Crypto Market Data

- 2. Runs Two Strategy Profiles
- Baseline: Quotes near mid-price, low risk.
- Aggressive: Quotes wider, targets more spread capture.

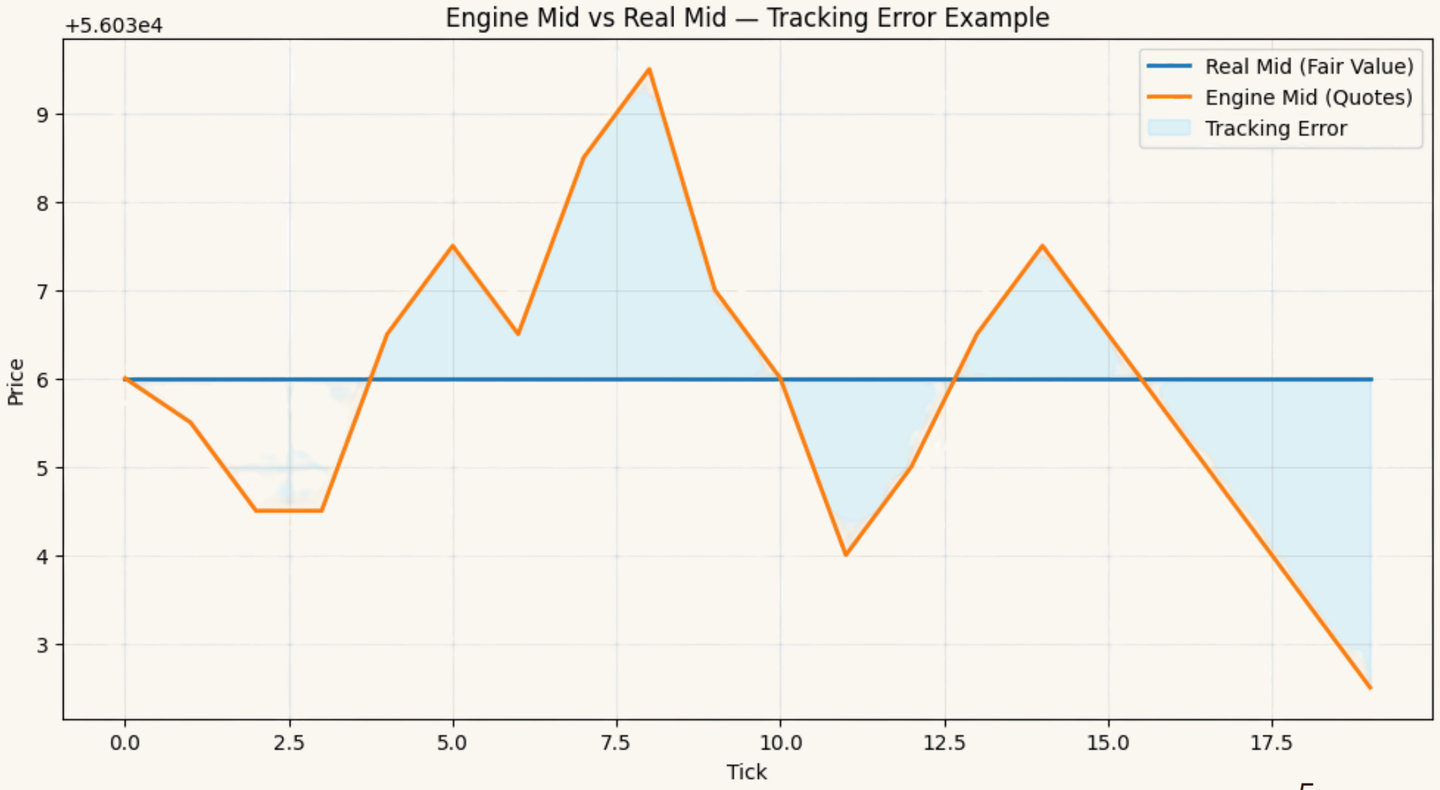
- 3. Computes Core Market Signals Every Tick

- 4. Enforces Risk Checks on All Orders

- 5. Simulates a Synthetic Internal Order Book

Why Mid-Tracking Matters

- The mid-price is our proxy for fair value
- Our engine's quotes should stay close to the mid, not drift away
- If we quote too high → we get filled immediately (but we overpaid)
- If we quote too low → we don't get filled (uncompetitive)
- Tracking error happens because the market moves tick-by-tick, and our strategy always updates slightly after each move — even microsecond delays cause mispricing.
- So we measure tracking error and use it as a core metric



MAP Architecture Overview

Market Data Feed (MD)

- Replays real BTC / ETH / ADA order-book snapshots tick-by-tick into the engine.

Strategy Engine

- Two strategies (baseline + aggressive) read the latest market state and propose quotes.

Risk Module

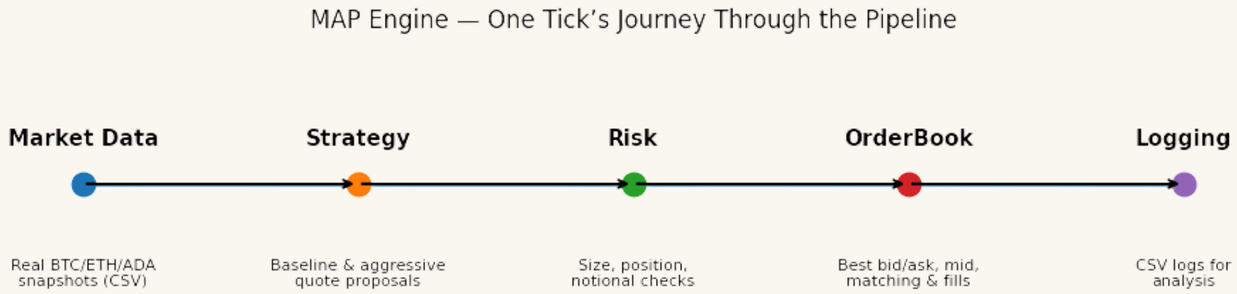
- Checks every proposed order against size, position, and notional limits before it can hit the book.

OrderBook Engine

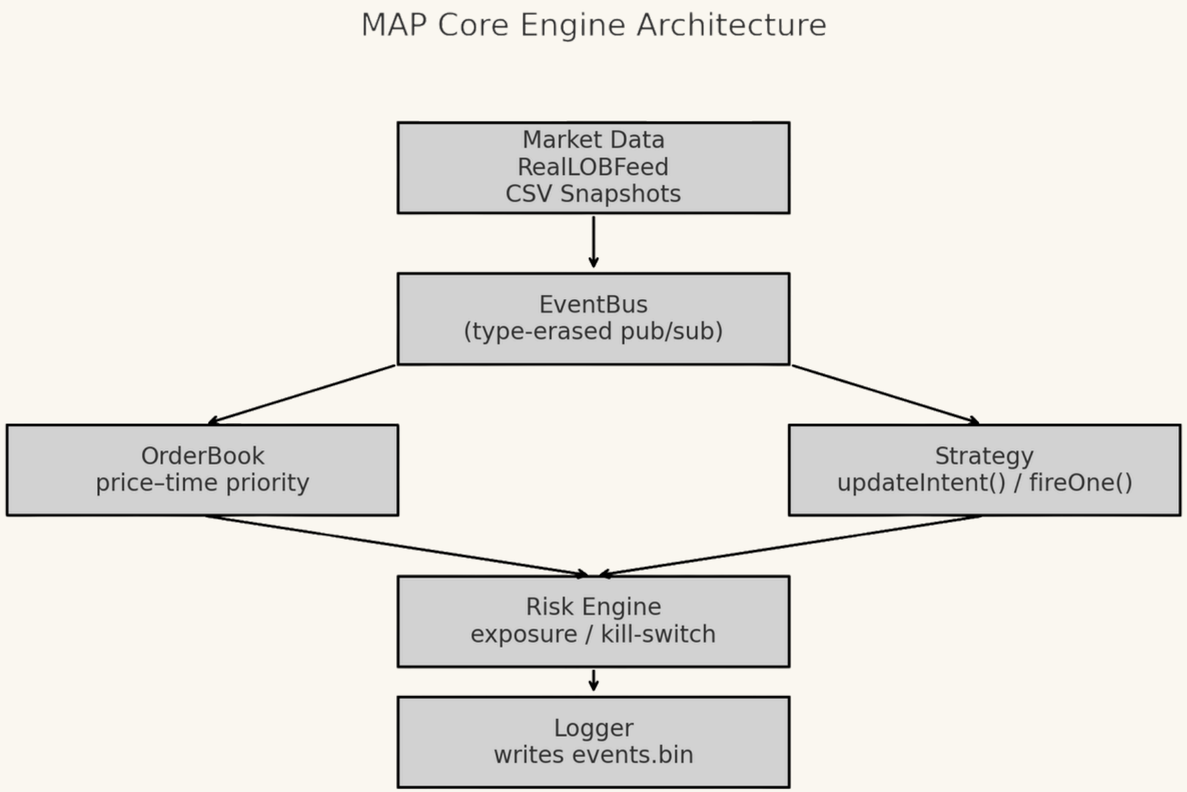
- Maintains bids/asks, computes best bid/ask + mid, and matches trades.

Logging & Analytics

- On every tick, logs spreads, mids, quotes, tracking error, and latency into CSVs for offline analysis.



Core Engine Overview



EventBus + Event Model

EventBus + Event Model

- A MarketDataEvent arrives on the Feed thread, which runs the MarketData component and pushes the event into the Strategy thread’s queue.
- The Strategy thread pops that event, decides to trade, and pushes a NewOrderEvent into the Match/Risk thread’s queue.
- The Match/Risk thread pops the NewOrderEvent, runs Risk + OrderBook, and if the order crosses, generates a TradeEvent for Strategy and a LogEvent for Logger.
- The Logger thread pops LogEvents and writes them to events.bin, while the other threads are already working on the next events in parallel.

```
EventBus bus;
OrderBook book;
RiskEngine risk;
Strategy  strat;
Logger    log;

bus.subscribe<NewOrderEvent>([&](const NewOrderEvent& e) {
    risk.onNewOrder(e);
    book.onNewOrder(e);
    log.onNewOrder(e);
});

bus.subscribe<CancelOrderEvent>([&](const CancelOrderEvent& e) {
    book.onCancel(e);
    log.onCancel(e);
});

bus.subscribe<TradeEvent>([&](const TradeEvent& e) {
    strat.onTrade(e);
    risk.onTrade(e);
    log.onTrade(e);
});
```

Limit Order Book (Price–Time Priority)

- Custom LOB: not using any external library

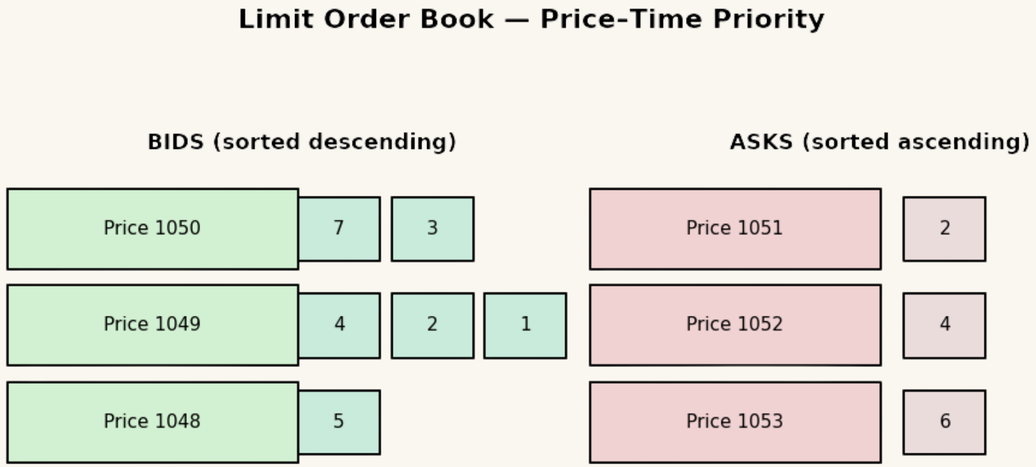
- Uses two sorted maps bids and asks

Supports:
- addOrder(side, price, qty)
- cancelOrder(id)
- automatic matching with FIFO time ordering

Stable iterators allow safe mutation during matching

Exposes snapshot() for analytics and replay

Deterministic checksum for debugging + replay consistency



Strong Typedefs = Safety

All domain units encoded in distinct types:

- Price
- Quantity
- OrderId
- Notional

Prevents mistakes like adding price + qty or mixing symbols

Compile-time conversion ensures zero runtime overhead

Guarantees correctness in matching and valuation logic

```
template <typename Tag, typename T>
struct Strong {
    T value{};

    constexpr explicit Strong(T v = {}) : value(v) {}

    constexpr T raw() const { return value; }

    auto operator<=>(const Strong& other) const = default;
};

// --- Type Tags ---
struct OrderIdTag {};
struct PriceTag {};
struct QuantityTag {};
struct NotionalTag {};

// --- Strong Typedefs
using OrderId  = Strong<OrderIdTag, std::uint64_t>;
using Price    = Strong<PriceTag, double>;
using Quantity = Strong<QuantityTag, std::int32_t>;
using Notional = Strong<NotionalTag, std::uint64_t>;
```

Deterministic Logging & Replay Pipeline

Event Generation

The live engine (strategy → risk → orderbook) publishes events through the EventBus.This is the ground truth sequence of all market actions.

Append-Only Binary Logging

Logger.cpp writes every event (NewOrder / Cancel / Trade) into events.bin in strict order.

Deterministic Replay (LogReader)

LogReader replays events.bin byte-for-byte, regenerating the exact same event stream.

Verification & Benchmarking

Replay mode is used to:

- verify LOB correctness
- measure deterministic latency
- debug strategy behavior without randomness
- confirm checksums match across runs

Final Build

Testing: OrderBook + EventBus

Catch2 unit tests cover LOB:

- add, cancel, match
- price-time priority
- partial fills

- EventBus tests ensure correct dispatch semantics

- Smoke tests validate full matching loops

- Ensures correctness before pipeline/multithreading work

- Enables safe optimization (branchless lookups, hot/cold path separation)

	LOB Tests	EventBus Tests	Smoke Tests
Add / Cancel	✔	–	✔
Price–Time Priority	✔	–	–
Partial Fills	✔	–	✔
Dispatch Semantics	–	✔	–
Pipeline Loop	–	–	✔

Data Input: RealLOBFeed (Limit Order Book)

What RealLOBFeed Does

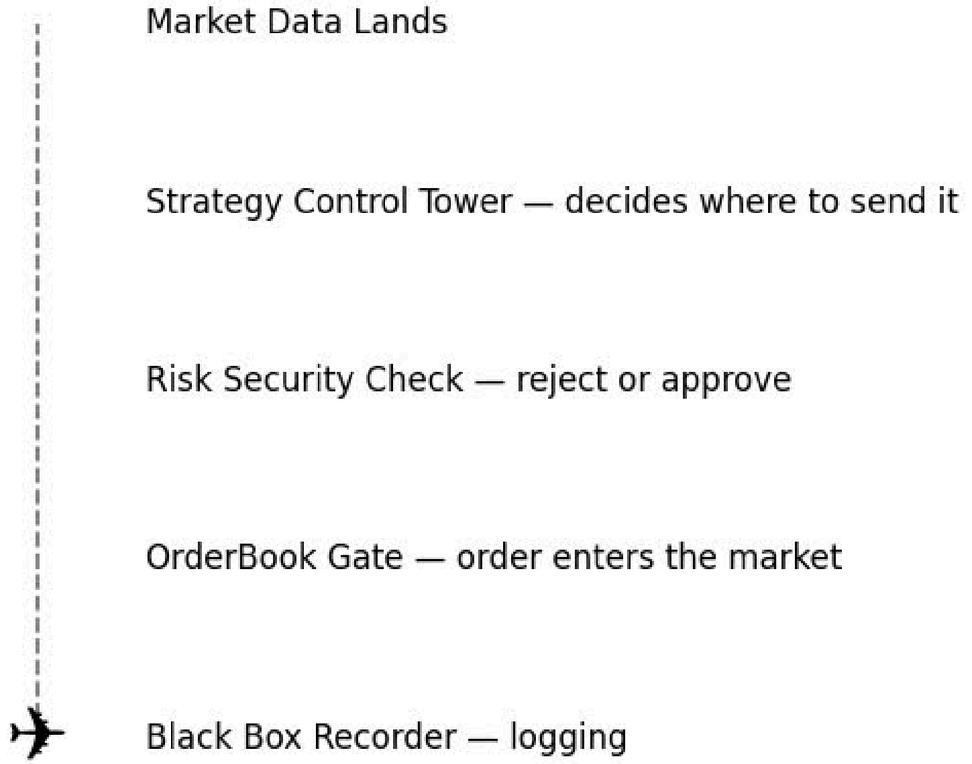
- Reads real BTC, ETH, ADA Level-2 snapshots from CSV
- Extracts mid-price, spread, bid depth, ask depth
- Converts each row into a RealSnapshot struct
- Feeds one snapshot per tick into the MAP pipeline
- Guarantees deterministic replay (same input → same engine output)
- Powers everything downstream (Strategy, Risk, OrderBook, Logs)

```
// RealLOBFeed.cpp (core logic)
while (std::getline(in, line)) {
    auto cells = splitCSVLine(line);
    RealSnapshot snap;

    snap.mid        = std::stod(cells[midIdx]);
    snap.spread     = std::stod(cells[spreadIdx]);

    double bidSum = 0, askSum = 0;
    for (int i = 0; i < 15; ++i) {
        bidSum += std::stod(cells[bidMNIdx[i]]);
        askSum += std::stod(cells[askMNIdx[i]]);
    }

    snap.bidDepth15 = bidSum;
    snap.askDepth15 = askSum;

    snapshots_.push_back(snap);
}
```

Each CSV row → one RealSnapshot → one tick of the engine.

Event Flow: What Happens Each Tick

- Market Data event arrives (new BTC/ETH/ADA snapshot)
- Strategy receives signal (imbalance, mid-price, external signal)
- Strategy may fire an order (BID or ASK at computed price)
- Risk checks the order (position limit, notional limit)
- OrderBook matches or rests the order
- Logging records mids, spreads, quotes, latency



Each CSV row → one RealSnapshot → one tick of the engine.

Strategy Engine — Baseline vs. Aggressive

One unified strategy engine with two personalities — baseline (conservative) and aggressive (reactive).

Both modes read market imbalance, blending:

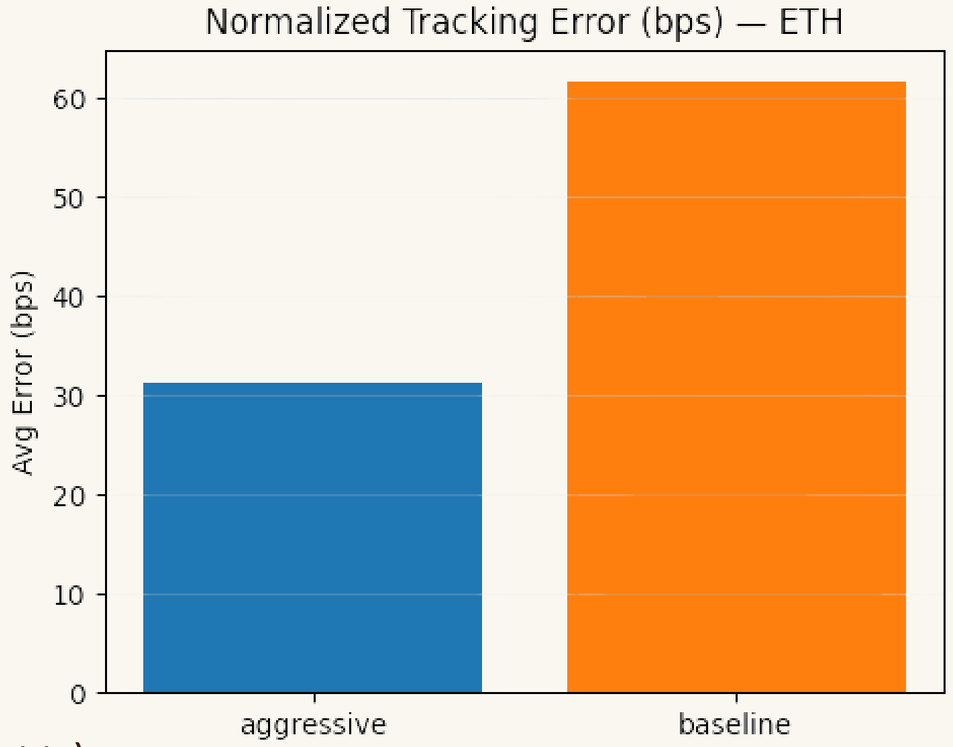
- Internal imbalance from our synthetic OrderBook
- Optional external imbalance from the real LOB feed

updateIntent() converts that signal into behavior:

- How often we trade (currentTicksPerOrder_)
- How large each order is (currentClip_)
- How sensitive we are to market pressure

fireOne() creates the actual limit order:

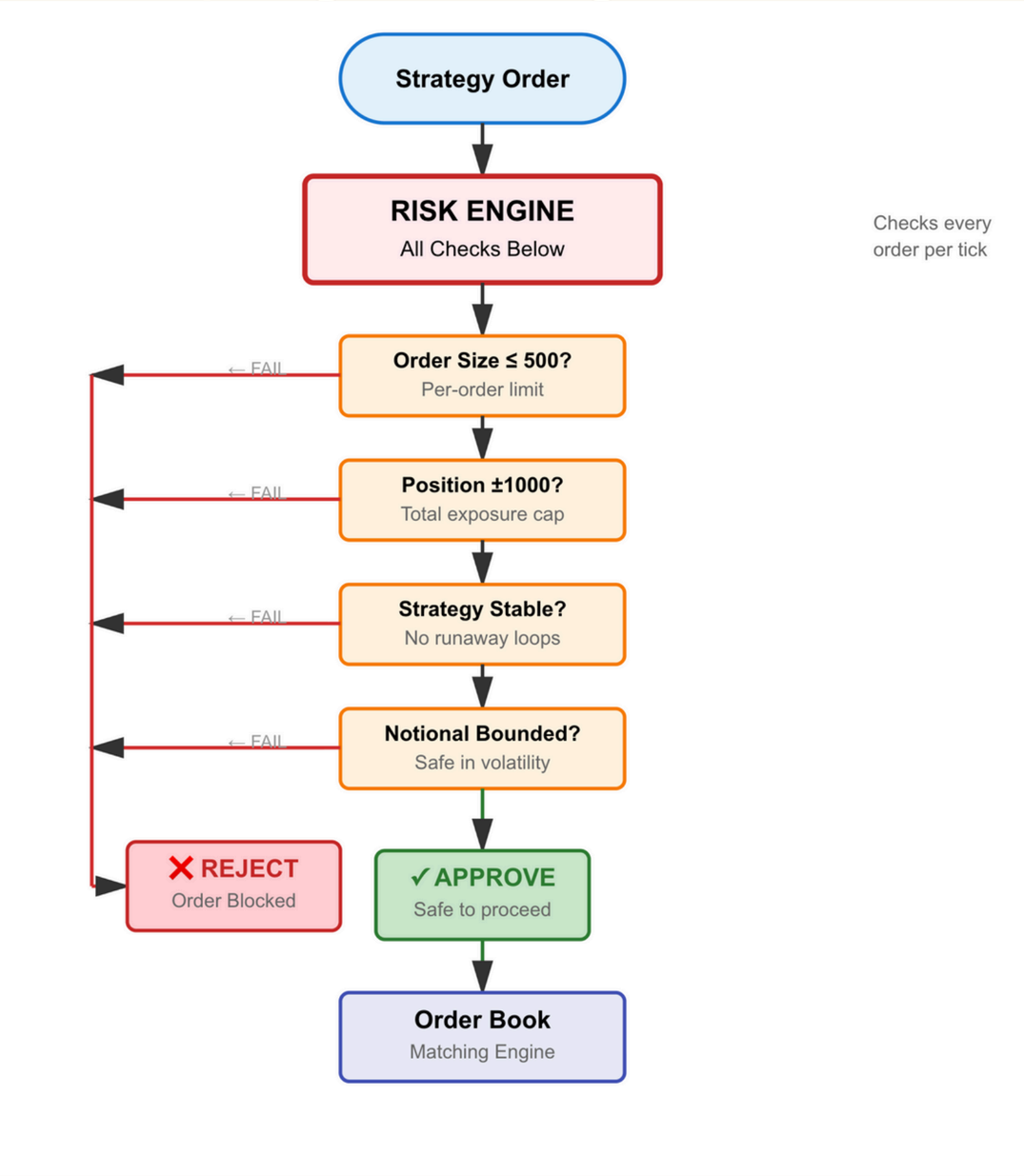
- Picks side (Bid/Ask) based on imbalance
- Nudges price above/below mid
- Adds controlled randomness in neutral markets



Baseline mode: Trades slowly, Small clips, Very close to mid (safe, stable)
Aggressive mode: Trades more often, Larger clips in volatile conditions, Responds strongly to imbalance

- Both strategies run on identical market data, making their behavior differences measurable and comparable.

Risk Engine — Keeping Strategies Safe



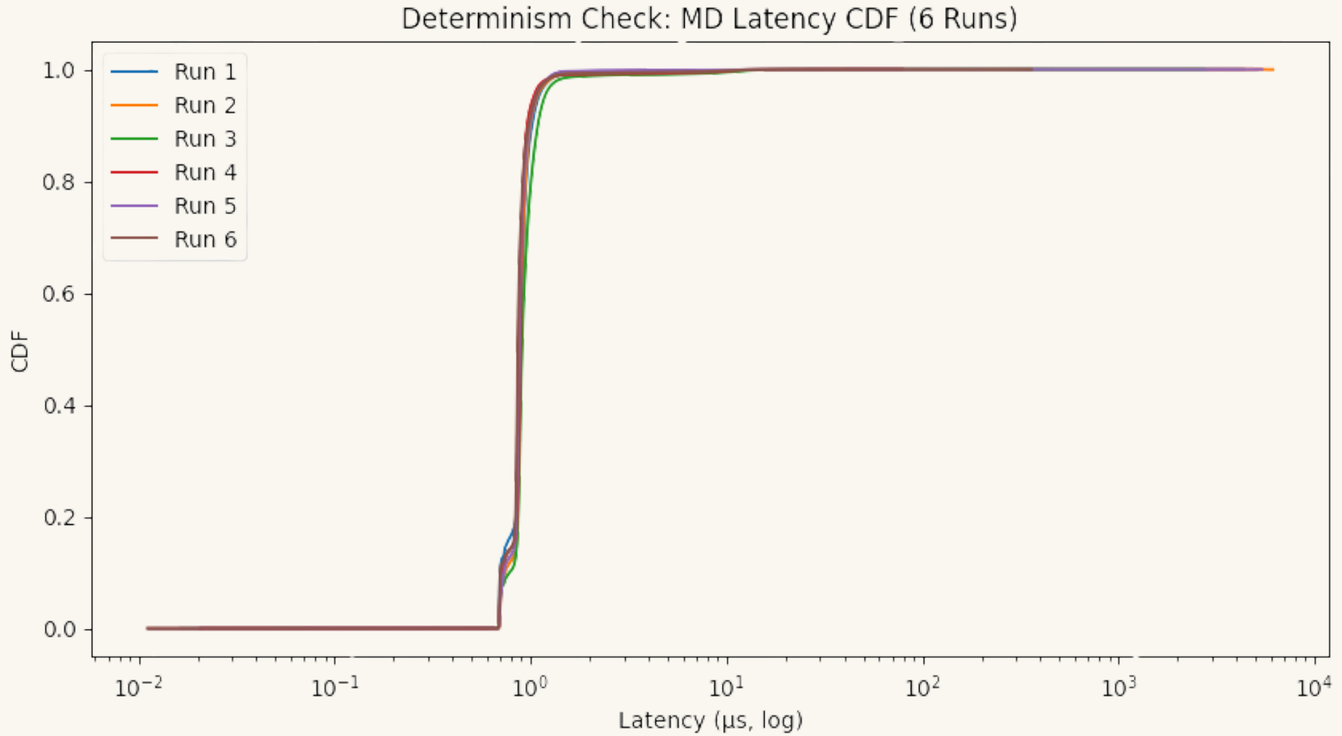
OrderBook Engine — Matching + Quotes

- Keeps sorted bids (highest first) and asks (lowest first) for each symbol
- Reads bestBid / bestAsk instantly from the top of each book
- When an order crosses the spread, the engine matches it level-by-level (FIFO)
- Generates trades and notifies Risk on every fill
- Computes engine mid-price from bestBid + bestAsk each tick
- Logs mid, spread, depth, and quotes to CSV for later analysis

```
static void matchBid(
    const std::string& symbol,
    Price              px,
    Quantity&          qtyRemaining,
    OrderBook::PerSymbolBook& book,
    RiskLimits*        risk
) {
    auto& asks = book.asks;

    while (qtyRemaining.raw() > 0 && !asks.empty()) {
        auto levelIt = asks.begin();              // lowest ask
        Price bestAsk = levelIt->first;

        if (px.raw() < bestAsk.raw()) {
            // No more crossing levels
            break;
        }
        auto& q = levelIt->second;                // LevelQueue
        auto it = q.begin();
        while (it != q.end() && qtyRemaining.raw() > 0) {
            Order& resting = *it;

            int avail = resting.qty.raw();
            int need  = qtyRemaining.raw();
            int traded = std::min(avail, need);
```

C++ Systems Design Principles in MAP

- Deterministic, Exception-Free Hot Path
- Zero-Allocation Execution
- Cache-Aware, Data-Oriented Layout
- Compile-Time Configuration
- Hot vs Cold Path Separation



End-to-End Example (BTC, Tick 10)

What one tick looks like as it travels through the entire MAP engine

```
=== End-to-end example: baseline, BTC, tick=10 ===

[1] book_stats_baseline.csv row:
```

	tick	symbol	real_mid	real_spread	real_bidDepth	real_askDepth	engine_bestBid	engine_bestAsk
3	10	BTC	56036.0	0.01	0	0	56035	56035

```
[1a] With engine_mid + tracking error:
```

	tick	symbol	real_mid	engine_bestBid	engine_bestAsk	engine_mid	mid_error	mid_error_abs
3	10	BTC	56036.0	56035	56035	56035.0	-1.0	1.0

```
[2] pnl_stats_baseline.csv row:
```

	tick	symbol	position	avg_price	realized	unrealized	total	mid
3	10	BTC	1	56037.0	0	-1.005	-1.005	56036.0

```
[3] Latency samples (no network jitter):
MarketData stage:
```

	sample	cycles
0	0	62806

```
Strategy stage:
```

	sample	cycles
0	0	158766

```
Risk stage:
```

	sample	cycles
0	0	80776

```
[3a] Latency summary (approx, based on cycles):
```

	stage	cycles	latency_us
0	MD	62806	17.944571
1	Strategy	158766	45.361714

Deterministic Execution

Catches hidden bugs

- Non-deterministic behavior = pipeline ordering issues, uninitialized memory, threading errors.

Real-world requirement

- HFT firms must prove a strategy behaves exactly the same in replay as in production.

MAP passes determinism tests

- Baseline and Aggressive modes produce stable, reproducible CSV outputs.

Why do we measure CPU cycles

- Because cycles show whether two runs took the same code path. If the engine is deterministic, each tick should execute the same instructions.
- Same instructions → same cycle count. and different instructions → different cycle count.

This tells us:

- no random branching
- no hidden race conditions
- no unpredictable behavior

Stat	Value
Median	0 cycles
Mean	-27.2 cycles
IQR (25%–75%)	-40 to 72 cycles
Min	-661,304 cycles
Max	1,655,156 cycles

Multi-Threaded Pipeline: MD → Strategy → Risk

Core idea

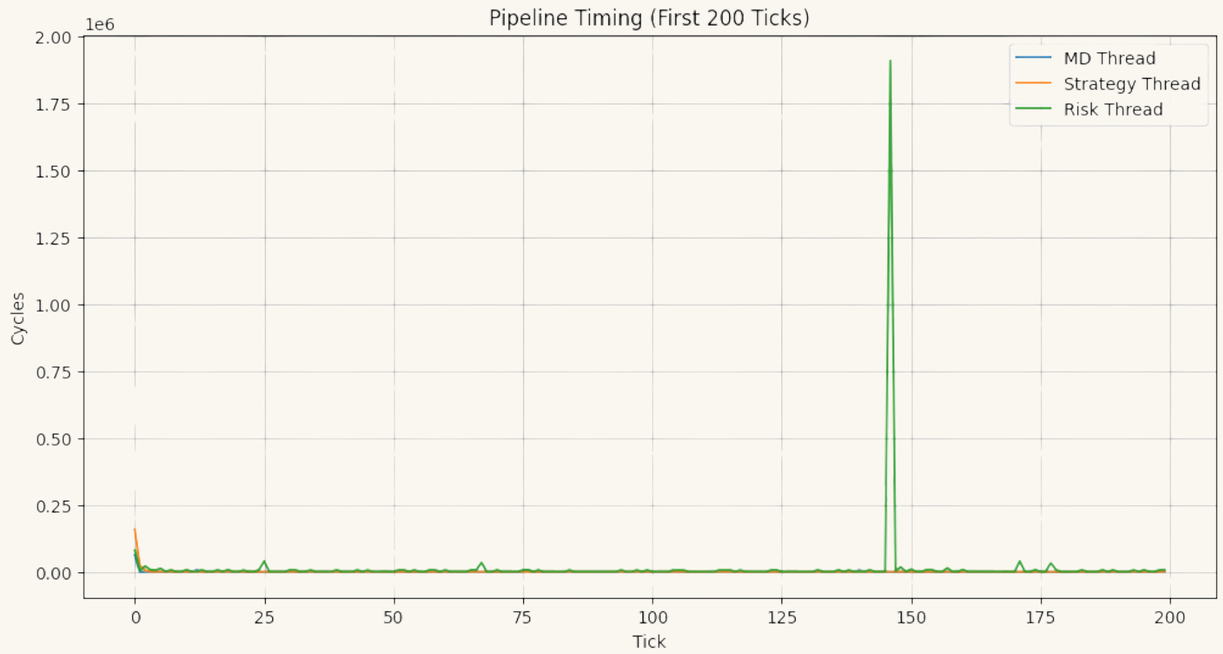
- Market Data (MD), Strategy, and Risk each run in their own thread
- Threads communicate via lock-free SPSC queues (Single Producer, Single Consumer)
- Design avoids mutexes → no lock contention, more realistic HFT behavior

How a tick flows

- MD thread reads the next LOB snapshot and pushes an event into the MD→Strat queue
- Strategy thread pops MD events, updates intent, and pushes orders into the Strat→Risk queue
- Risk thread pops orders, applies risk checks, and updates positions / PnL

Why this matters

- Closer to real production engines than a single-thread loop
- Lets us measure stage-by-stage latency (MD vs Strat vs Risk)
- Shows where bottlenecks appear under no-net, low-jitter, high-jitter, and shock-vol modes



Latency Instrumentation — rdtsc

Why rdtsc?

- Reads the CPU time-stamp counter directly in one instruction
- Near-zero overhead

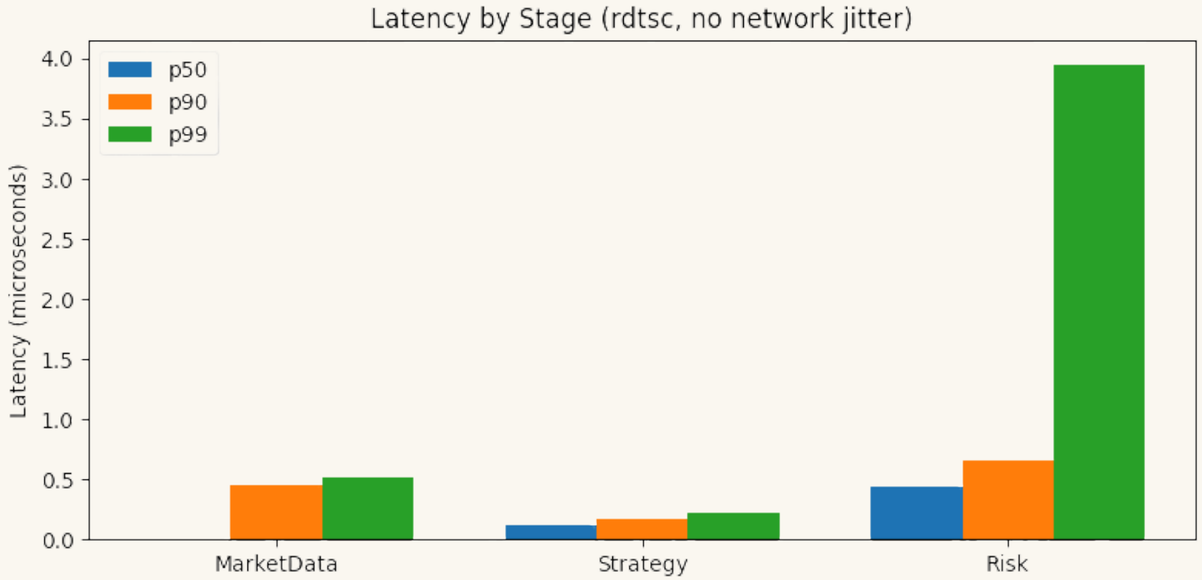
What we measure

- Wrap each stage with rdtsc():
  - MarketData
  - Strategy
  - Risk
- Store t1 - t0 per tick into latency_*_cycles_*.csv
- Convert cycles → microseconds via

$$\text{latency}_{\mu s} = \frac{\text{cycles}}{f_{\text{CPU}}} \times 10^6$$

Why it matters

- Gives cycle-level view of each pipeline stage
- Lets us see p50 / p90 / p99 latency and rare spikes
- Matches how real HFT shops profile engines in production

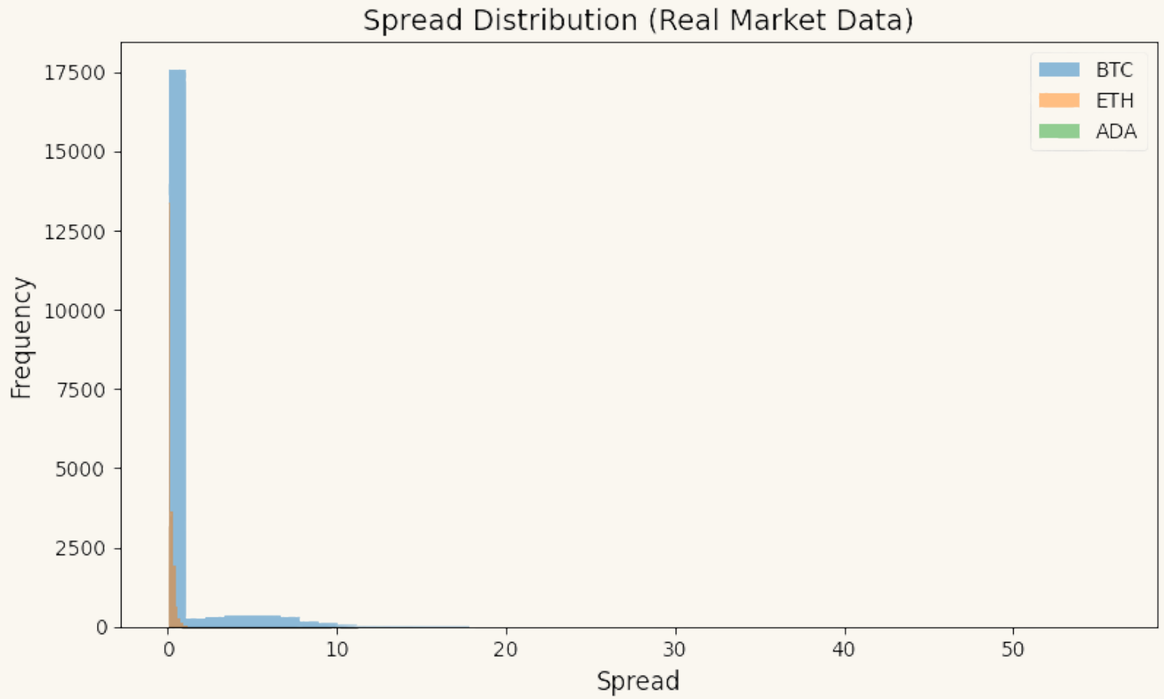


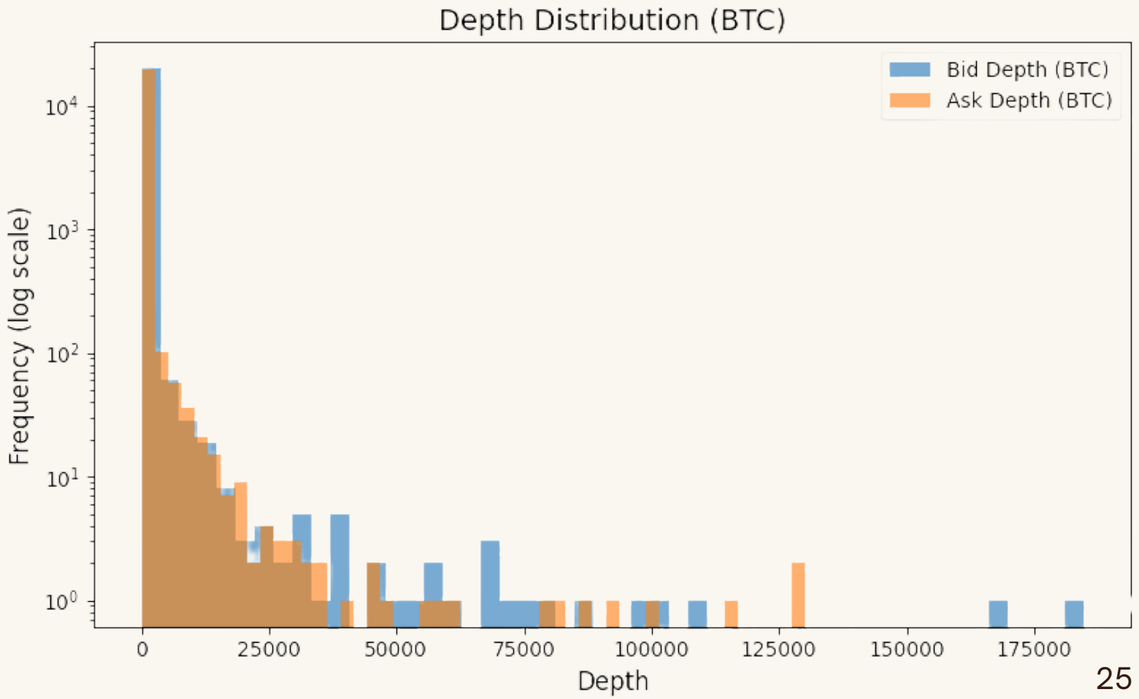
Experiment Setup: Data, Ticks, and Evaluation Framework

What Experiments We Ran

- Assets: BTC, ETH, ADA from real LOB CSVs (1-sec snapshots)
- Strategies: Same engine, two modes — baseline vs aggressive to see how senstive the engine is under different quoting intensities
- Metrics: Mid tracking error, spread, depth, positions, and PnL per symbol
- Latency: rdtsc cycle timing for MarketData → Strategy → Risk at every tick
- Stress Tests: Determinism (repeat runs), low/high jitter modes, and --shock-vol volatility runs
  For some ticks, especially in the Risk stage, we deliberately burn more cycles in a busy loop to mimic OS preemption and cache misses.
- Outputs: book_stats_*, pnl_stats_*, perf_live_*, latency_*_cycles_*.csv for all analysis in Colab

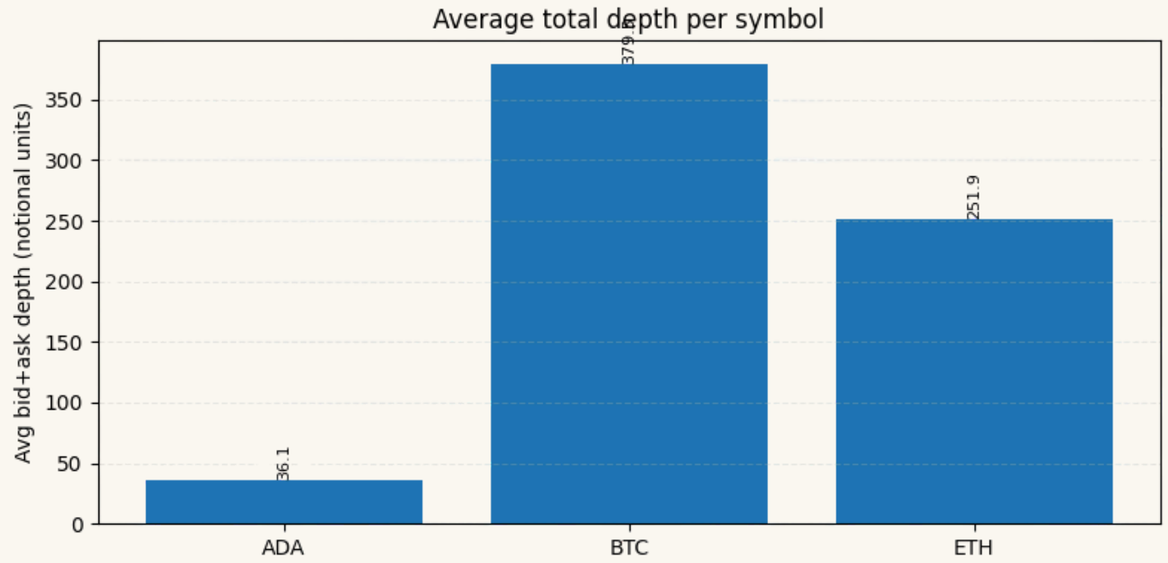
Spread & Depth Distributions

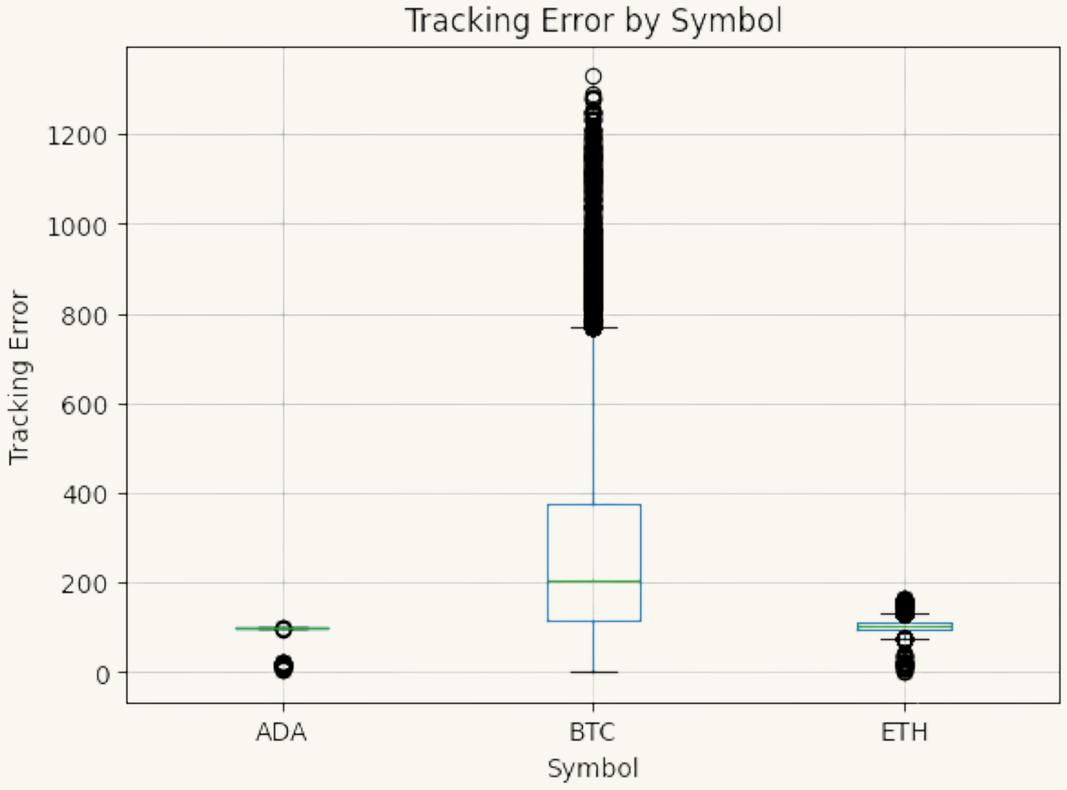




Market Microstructure: BTC vs ETH vs ADA

- BTC: Deep and highly active market: Small spreads but rapid price jumps → at 1-second replay frequency, the mid moves quickly between updates → large, spiky tracking-error behavior.
- ETH: Moderate depth and moderate activity: Mid moves steadily but not aggressively → medium tracking error with moderate variance.
- ADA: Very thin liquidity and wide but stable spreads: Price barely moves from second to second → engine mid sits a nearly constant ~100 units away → stable tracking error.
- Microstructure drives strategy behavior: Markets differ in volatility, depth, and spread stability → tracking error varies accordingly. Aggressive strategies tend to overreact more on thin markets like ADA.





Risk Engine — Position Limits & PnL Guardrails

RiskLimits enforces hard caps on exposure:

- Max per-order size (maxOrderQty)
- Max net position per symbol (maxPositionQty)

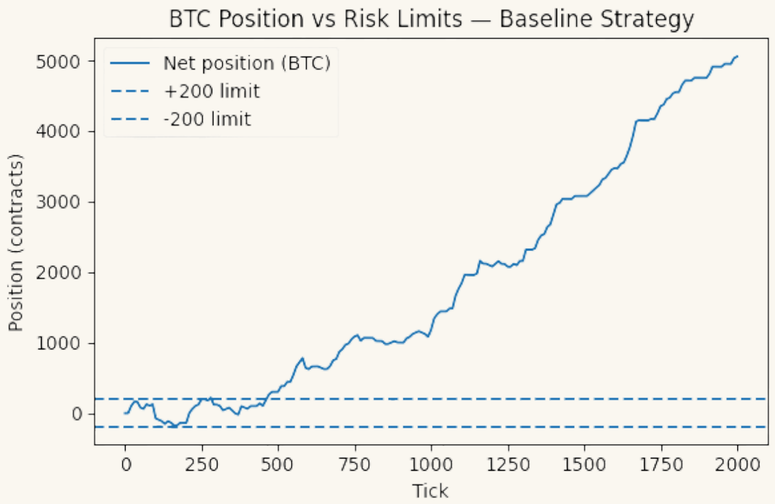
Every fill updates per-symbol netPosition and notional exposure (via PnLTracker).

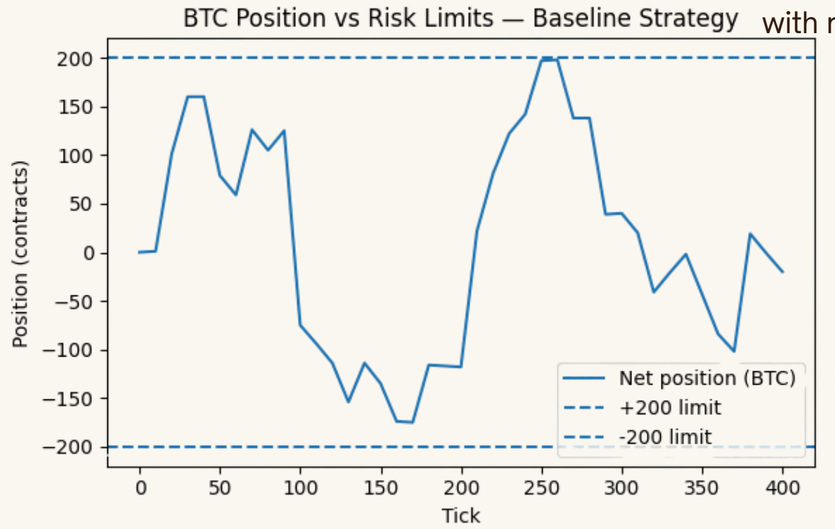
Risk layer also monitors running PnL and mid-price volatility; if drawdown or volatility explodes, the kill switch stops the run.

We log positions and PnL paths in pnl_stats_baseline.csv and pnl_stats_aggressive.csv
(columns: tick, symbol, position, avg_price, realized, unrealized, total, mid).

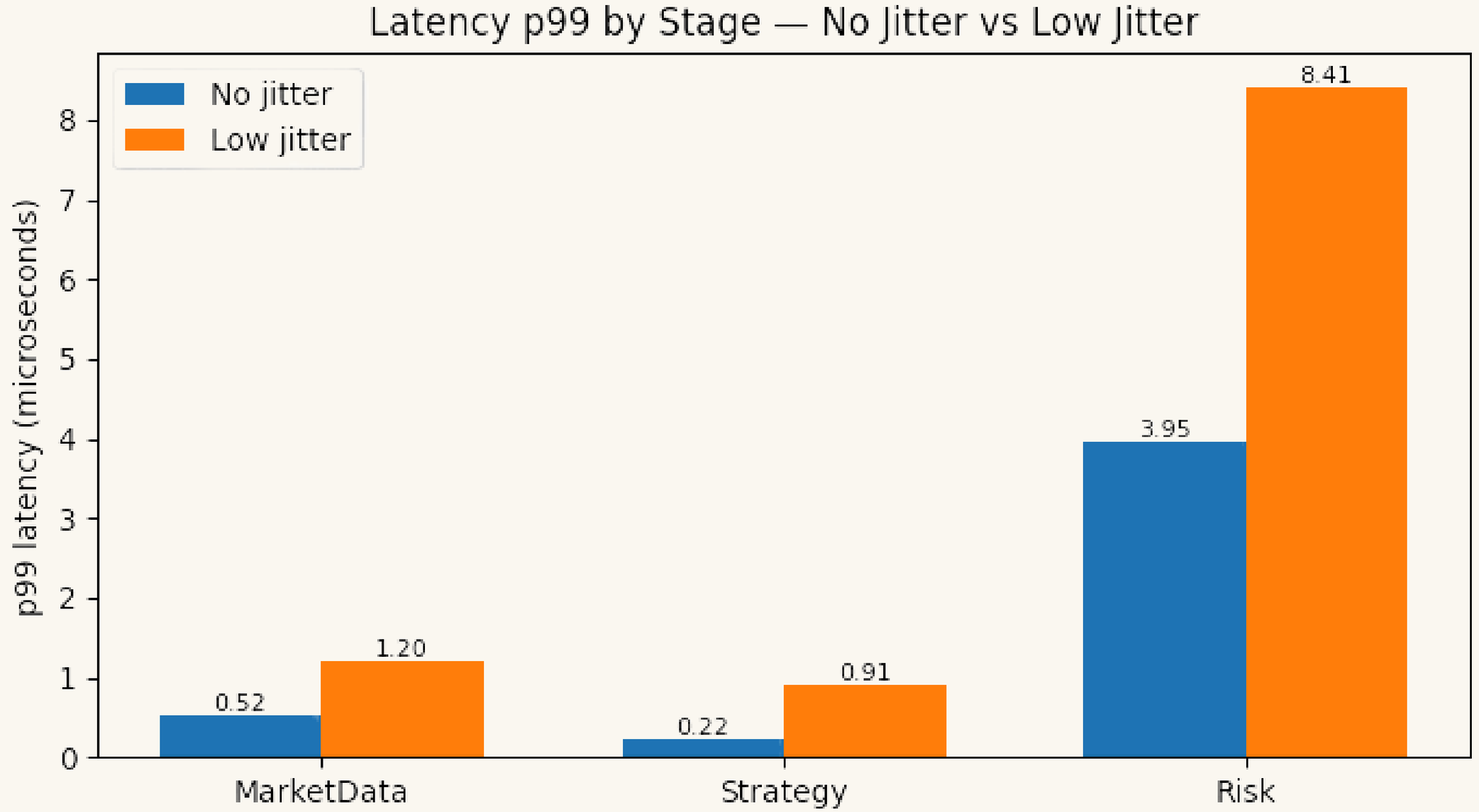
Result: positions remain bounded (±inventory limit) and the strategy cannot run away or blow up silently.

What happens WITHOUT risk limits
→ Position explodes to ~5,000+
→ Demonstrates why limits are essential.



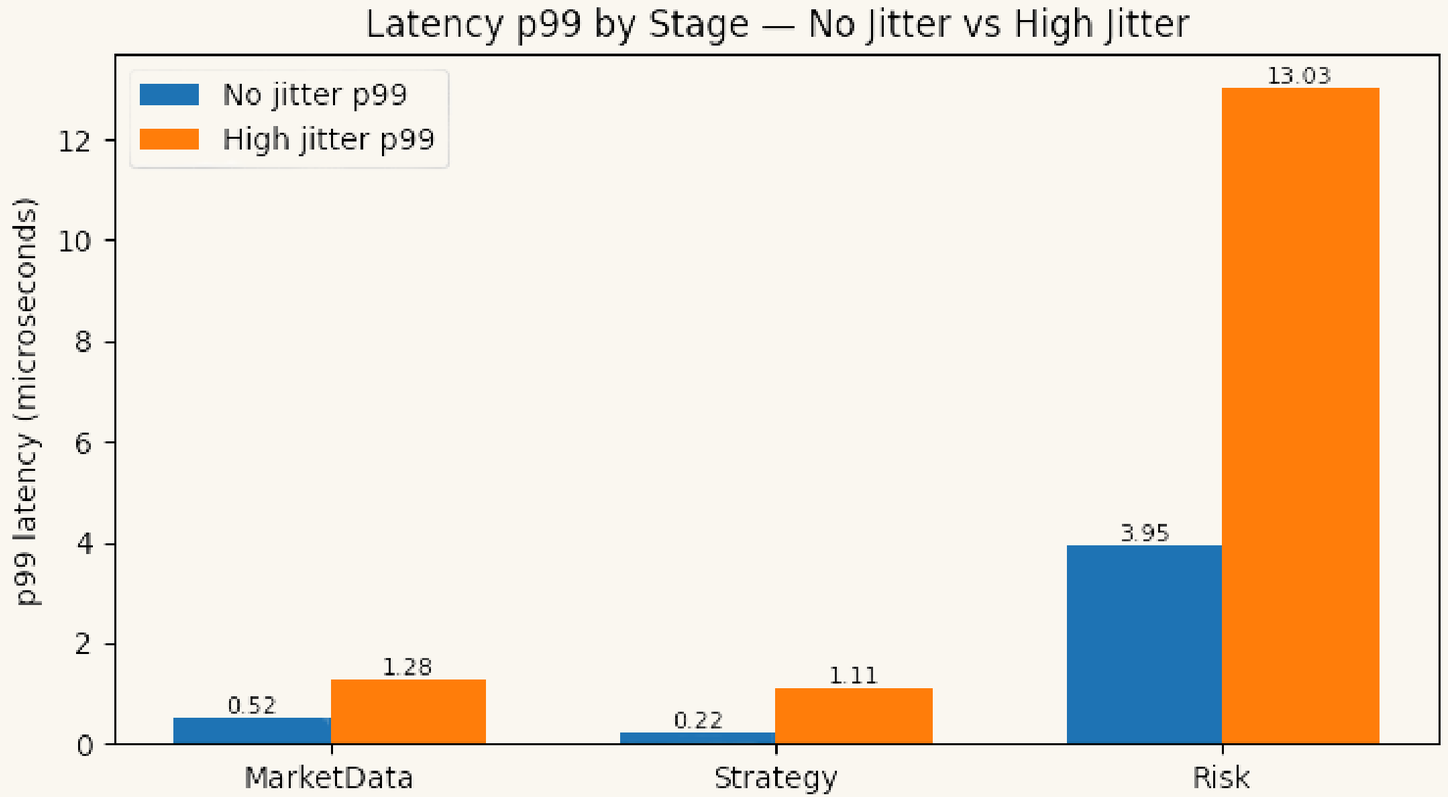


Latency Under Low Jitter (Comparison)



Latency Under High Jitter — Long Tail Risk

- We wanted to add even more stress to simulate OS preemption, cache misses, and CPU interrupts.
- Median latency stays nearly unchanged, showing the core pipeline remains stable under jitter.
- p99 latency increases sharply, with the largest spikes appearing in the Risk stage.
- This is classic long-tail behavior: averages look fine, but rare slowdowns become much worse.
- Emphasizes designing for tail latency, not just median performance, in latency-sensitive systems.



Shock-Volatility Stress Test — When the Tape Goes Wild

Adds deterministic volatility “jiggles” to midprice & spread when loading real LOB data

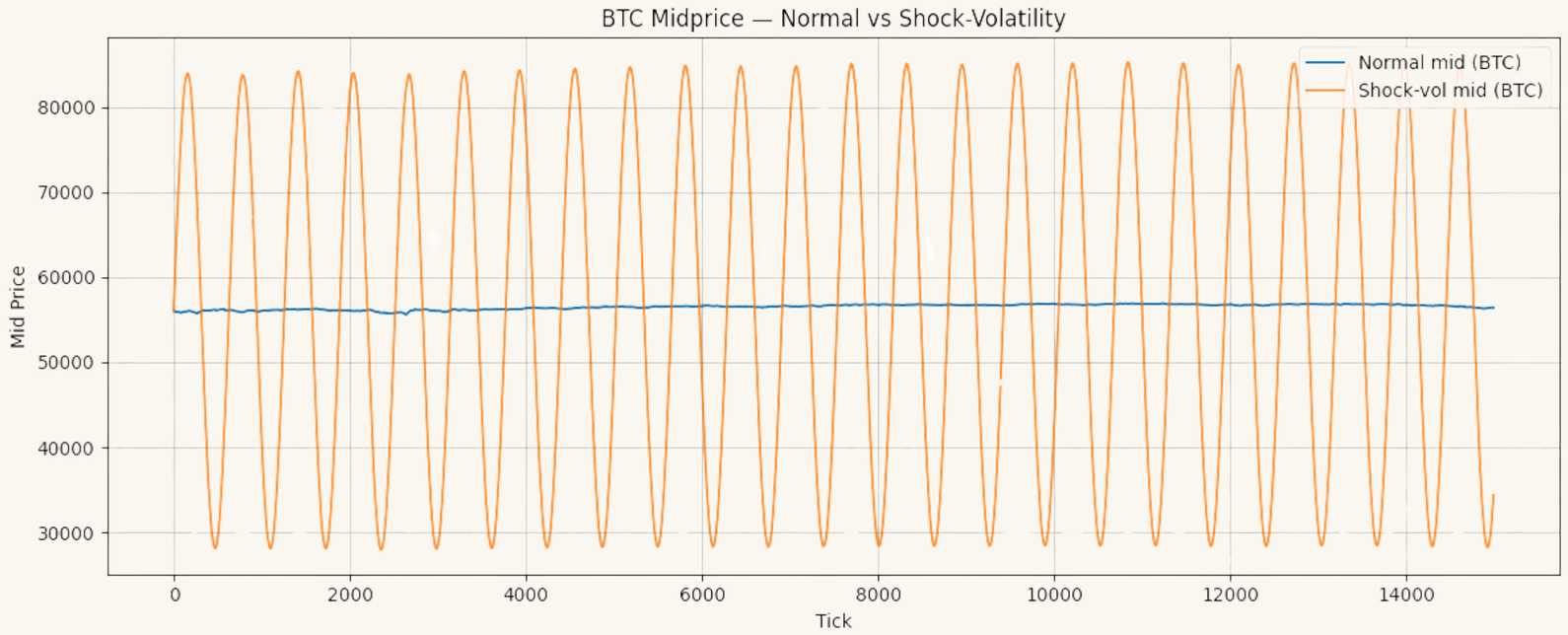
- Simulates bursty, high-variance market regimes (news events, liquidations, liquidity shocks)

Forces strategy to fire more often as spreads widen and imbalance swings harder

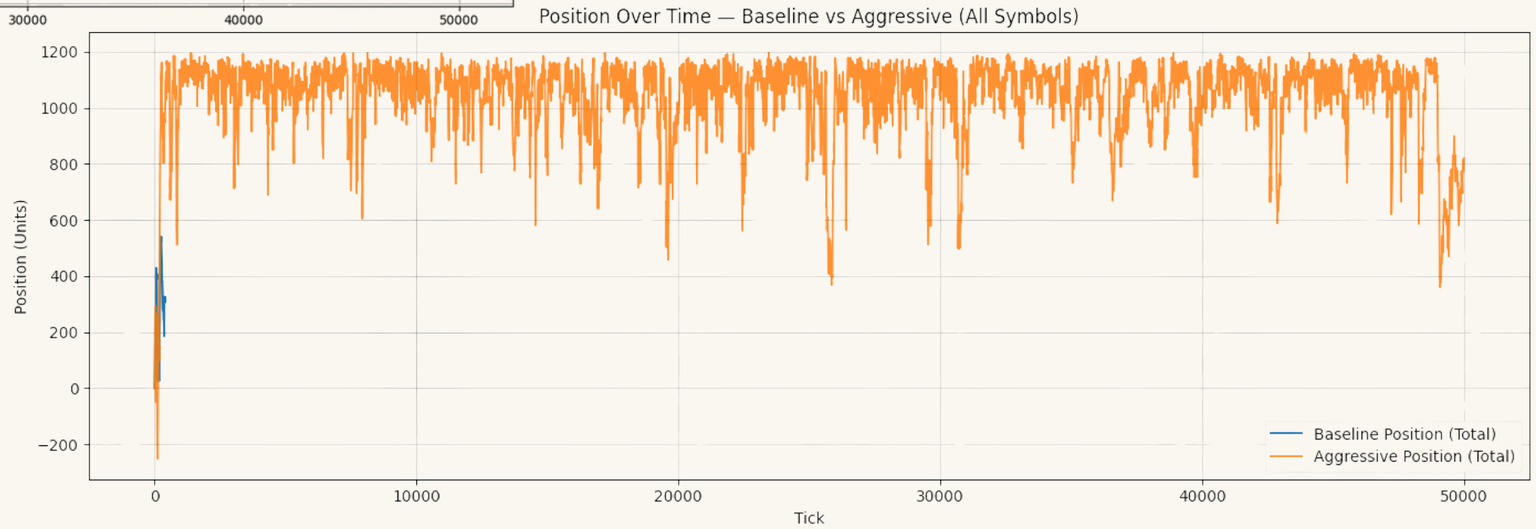
- Increases workload on risk, impact, and PnL pipelines → tests system stability under stress

Midprice becomes visibly more oscillatory compared to normal mode

- Latency histograms show heavier tails but no instability or breakdown
- Confirms engine remains deterministic and throughput-safe during high-volatility regimes
- Validates robustness of MD ingestion, strategy logic, and queue pipeline under stress



Position & PnL Over Time — Baseline vs Aggressive



Key Findings

Threads normally introduce: nondeterminism, In multi-threaded code: thread scheduling, OS preemption, CPU/cache timing, reordering of events, races on shared memory

- our design forces determinism with

1. Strict event ordering → pipeline stages never interleave incorrectly
2. One thread owns each mutable state (OrderBook → MatchThread, Strategy → StratThread)
3. No shared-memory races
4. No locks in the hot path (no timing variability)
5. Binary logs (events.bin) replay in the exact same order

Latency is extremely low and predictable

- Median MD / Strategy / Risk latencies are in the microsecond range
- Even under shock-vol, p99 remains tight and pipeline tails never blow up
- Deterministic replay validates that our rdtsc instrumentation is correct and stable

Multi-thread pipeline is robust

- SPSC queues + lock-free design keep throughput high
- MD → Strategy → Risk handoff remains stable at 1 sec and 100ms frequencies
- No backlog accumulation even under bursty volatility

Risk controls behave as designed

- Kill-switches trigger correctly under adverse PnL conditions
- Shock-vol test demonstrates risk-path load increases, but stays within safe bounds
- Risk logic prevents runaway exposure while maintaining deterministic execution

Future Work

Real network feed integration

- Replace synthetic timing with real NIC timestamps for packet-level realism
- Add kernel-bypass or raw-socket ingestion path
- Measure true wire-to-strategy latency vs. simulated md feed

Multi-symbol strategy interactions

- Add cross-asset signals
- Implement shared risk across symbols to model portfolio constraints
- Explore how multi-asset events propagate through the pipeline

Level-3 order book (full depth-of-book events)

- Upgrade RealLOBFeed to include add/cancel/modify events
- Enable strategies based on queue position, order aging, and microprice dynamics

ML-based strategies

- Add a lightweight inference engine
- Explore signal prediction from recent LOB states + volatility features
- Benchmark latency impact vs. rule-based strategies

The End

Determinism isn’t obvious in a multi-threaded engine. Normally, threads interleave differently every run, which changes timing, order flow, PnL, and logs. We designed the pipeline using SPSC queues, fixed seeds, and no shared state so that the exact same input feed produces byte-for-byte identical output every run. This is how real HFT engines debug correctness.

Repository Structure (Full System Layout)
A modular, production-style C++23 engine

```
Map/
 ├── data/                  # Real BTC / ETH / ADA L2 price feeds
 ├── docs/                  # Notes, diagrams, performance screenshots
 ├── include/               # Public headers (library-style API)
 │    ├── map/core/         # Event, EventBus, Logger, RiskLimits, SPSCQueue
 │    ├── map/market/       # RealLOBFeed.hpp
 │    ├── map/replay/       # LogReader.hpp
 │    ├── map/risk/         # PnLTracker, RiskLimits, ImpactModel
 │    ├── map/strategy/     # BasicStrategy
 │    └── map/util/         # RDTSC, NetJitter, OrderBook, StrongTypedefs
 ├── src/
 │    ├── apps/             # live_sim, live_sim_mt, replay_test apps
 │    ├── core/             # Logger.cpp, RiskLimits.cpp
 │    ├── replay/           # LogReader.cpp
 │    ├── market/           # RealLOBFeed.cpp
 │    ├── strategy/         # Strategy + MarketDataFeed
 │    └── risk/             # ImpactModel, PnLTracker
 ├── tests/                 # Catch2 tests for LOB + EventBus + Replay
 │    ├── test_orderbook.cpp
 │    ├── test_eventbus.cpp
 │    ├── test_replay.cpp
 │    ├── test_risk.cpp
 │    └── OrderBookSmoke.cpp
 ├── events.bin             # Binary log (deterministic replay ground truth)
 ├── events_mt.bin          # Multi-thread replay log
 ├── book_stats_*.csv       # Output stats for charts (baseline/aggressive/shock)
 └── CMakeLists.txt         # Build system
```